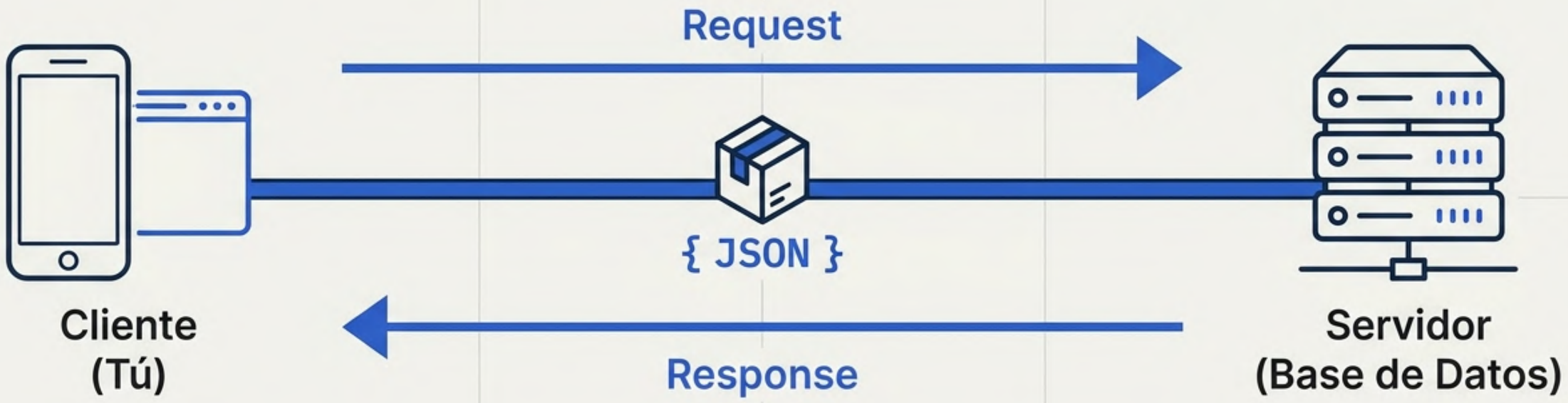
Conectando Sistemas con APIs REST

Una **API** es un puente que permite a dos sistemas comunicarse. **REST** es el estándar web basado en URLs (Endpoints).



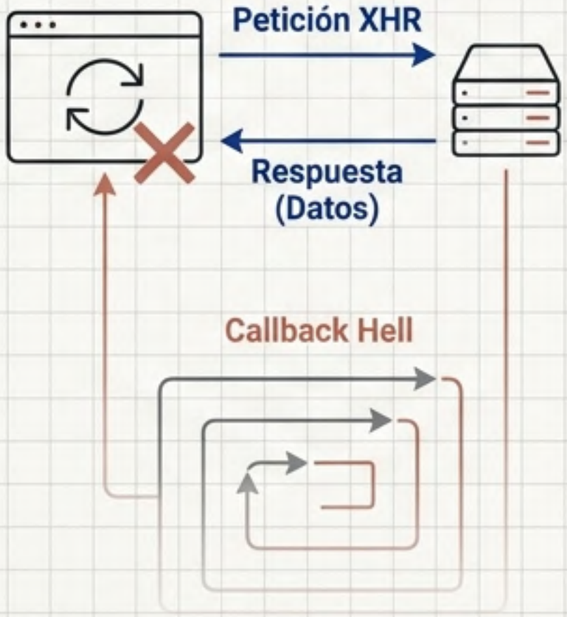

```
// Un Endpoint: La URL exacta a la que pedimos datos
const url = 'https://api.github.com/users/octocat';
```

NotebookLM

El Contexto Histórico: AJAX y XHR

AJAX resolvió el problema de recargar toda la página web para traer datos nuevos, utilizando el objeto **XMLHttpRequest (XHR)**.

Su sintaxis basada en eventos y callbacks era larga y propensa al desorden. Hoy es la base invisible de las **peticiones modernas**.




```
const xhr = new XMLHttpRequest();

xhr.open('GET', 'https://api.ejemplo.com/data');

xhr.onload = function() {
  console.log(this.responseText); // Callback hell
};

xhr.send();
```

NotebookLM

Fetch API: El Estándar Moderno

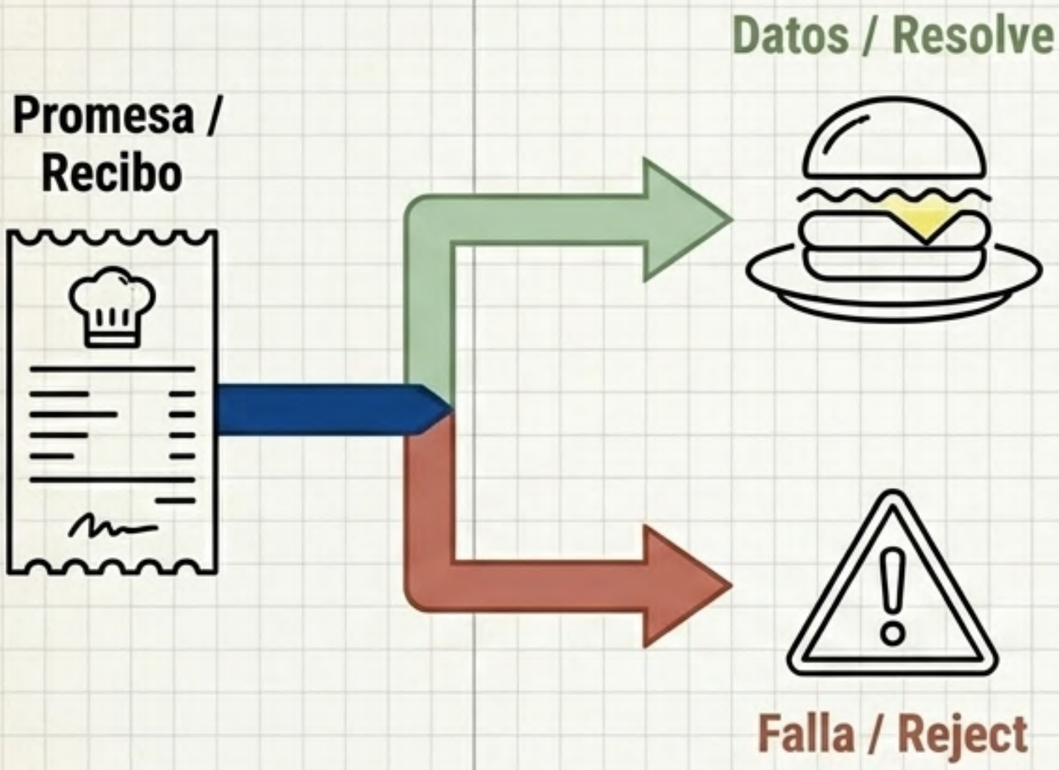
```
const url = 'https://jsonplaceholder.typicode.com/users';

// Iniciar la petición en segundo plano
fetch(url);
```

fetch() es la función nativa moderna para red.

Delega la tarea de red al navegador para no congelar la pantalla de la aplicación.

NotebookLM

Entendiendo las Promesas

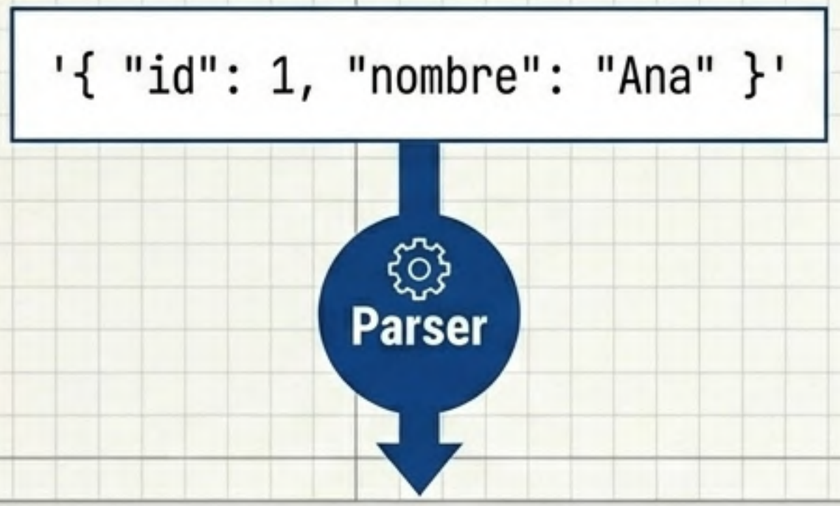


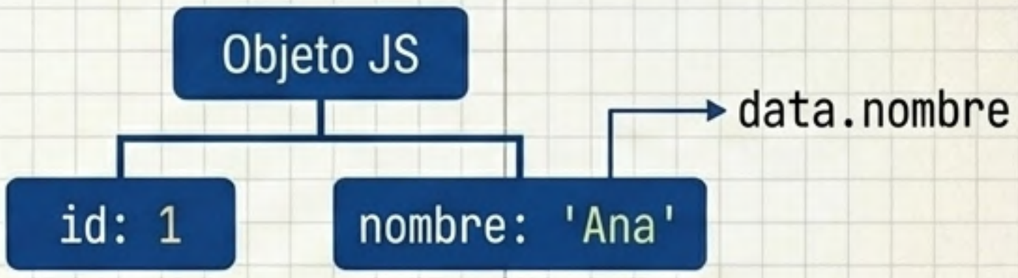

```
fetch(url)
  .then(response =>
    console.log('¡Éxito!', response))
  .catch(error =>
    console.error('¡Falló la red!', error));
```

Una Promesa es un objeto que representa el éxito o fracaso eventual de una operación asíncrona. Usamos **.then()** para procesar la respuesta y **.catch()** para atrapar errores de conexión.

NotebookLM

El Formato Universal: JSON

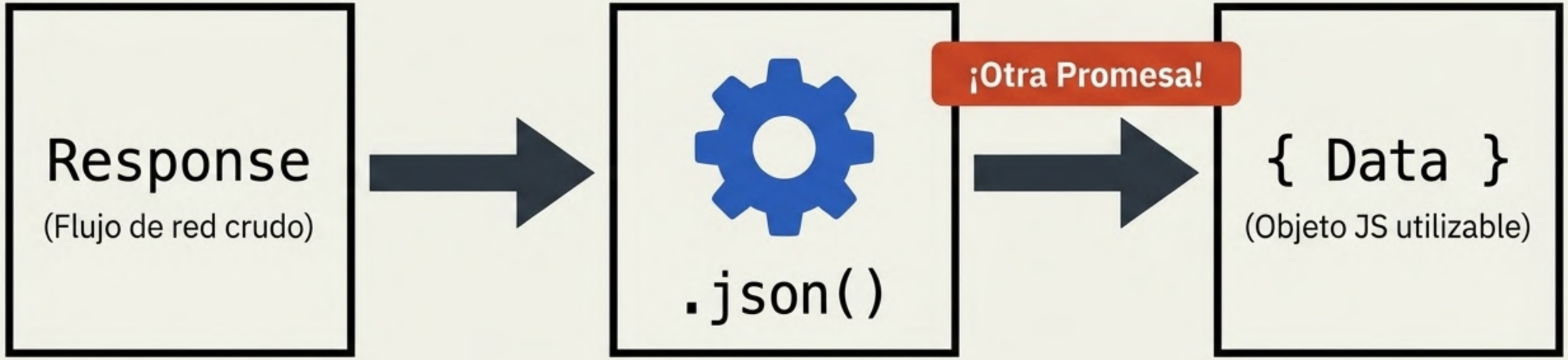





```
// Datos convertidos a Objeto nativo
const data = {
  id: 1,
  nombre: 'Ana',
  curso: 'Bootcamp JS'
};
console.log(data.nombre); // Resultado: 'Ana'
```

NotebookLM

El Flujo Fetch: Procesando la Respuesta




```
fetch('https://api.ejemplo.com/movies')
  .then(response => response.json()) // ⚠️ Devuelve OTRA promesa
  .then(data => console.log(data)); // Aquí ya tenemos los datos
```

NotebookLM

Recorriendo Colecciones de Datos

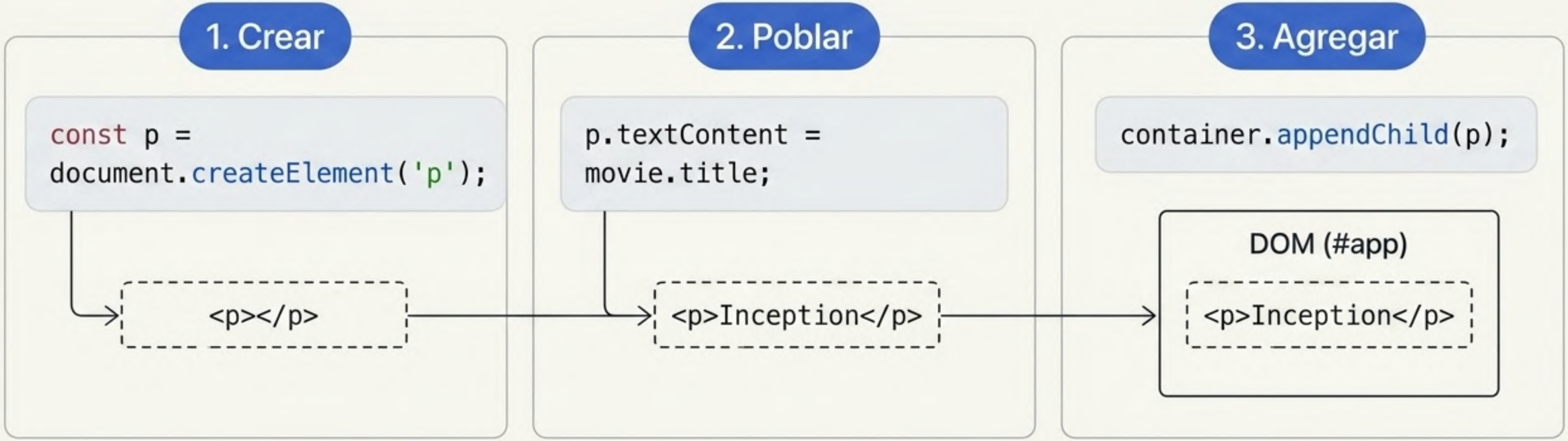
Las APIs devuelven Arreglos llenos de objetos. Usamos `for...of` para inspeccionar cada elemento.

```
fetch(url)
  .then(res => res.json())
  .then(movies => {
    for (const movie of movies) {
      console.log(`${movie.id}: ${movie.title} (${movie.year})`);
    }
});
```

```
> 1: Inception (2010)
> 2: The Matrix (1999)
> 3: Interstellar (2014)
```

NotebookLM

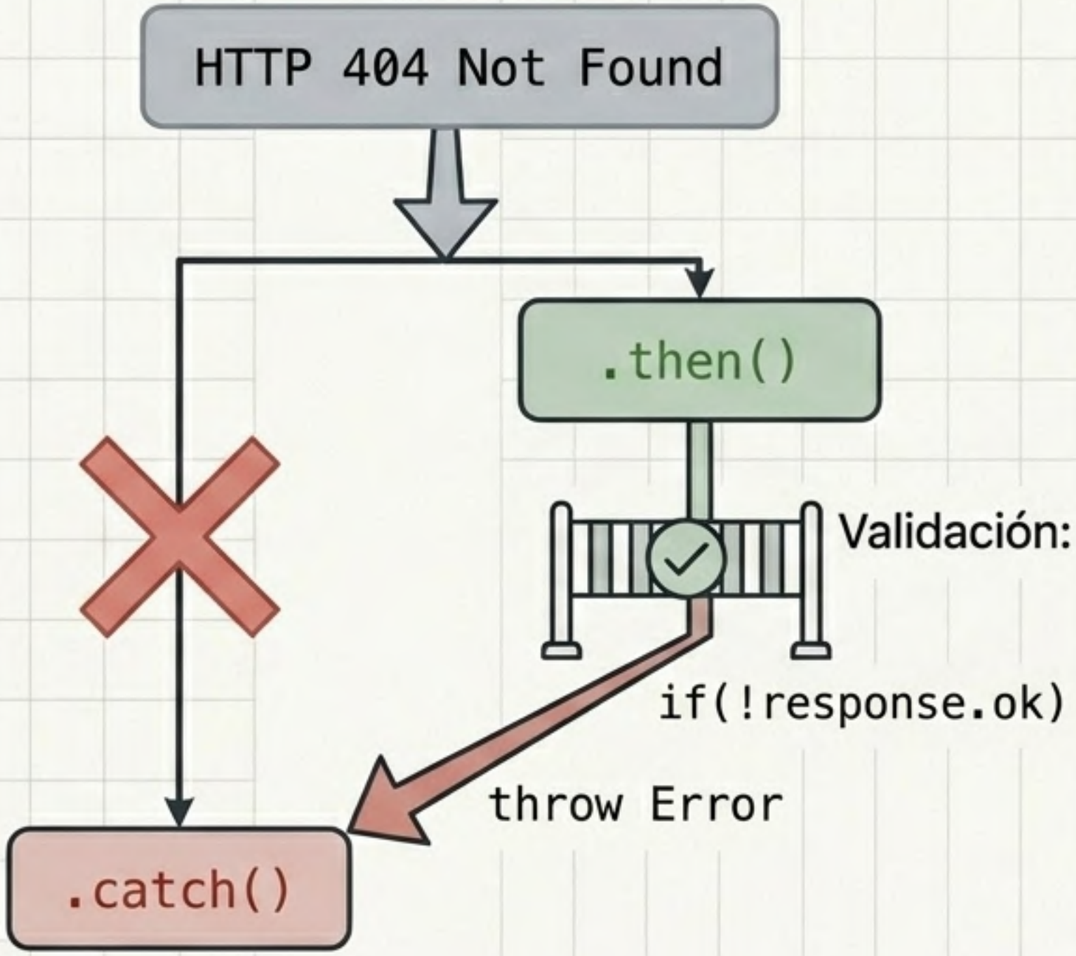
Renderizado Seguro en el DOM



Construimos el nodo en memoria y lo anclamos al documento. Esto previene ataques de inyección (XSS) **al evitar concatenar strings HTML.**

NotebookLM

La Trampa de Fetch: Manejo de Errores




```
fetch(url)
.then(response => {
  // ⚠️ Validación OBLIGATORIA
  if (!response.ok) {
    throw new Error(`Error HTTP:
          ${ {response.status}`);
  }
  return response.json();
}).catch(err =>
console.error('Fallo atrapado:', err));
```

NotebookLM

Async / Await: Sintaxis Moderna

Permite pausar la ejecución de una función hasta que los datos lleguen, haciendo el código lineal y fácil de leer.

```
async function obtenerDatos() {
  try {
    const response = await fetch(url);
    if (!response.ok) throw new Error('Fallo en API');

    const data = await response.json();
    console.log(data);
  } catch (error) {
    console.error('Error:', error);
  }
}
```

Pausa la función aquí

Manejo de errores clásico

NotebookLM

Comparación: .then() vs Async/Await

Ambas opciones hacen lo mismo por debajo. Elegimos async/await para evitar la indentación profunda.

Estilo Clásico (.then)

```
fetch(url)
  .then(res => {
    if(!res.ok) throw new Error();
    return res.json();
  })
  .then(data => console.log(data))
  .catch(err => console.error(err));
```

Estilo Moderno (await)

```
try {
  const res = await fetch(url);
  if(!res.ok) throw new Error();
  const data = await res.json();
  console.log(data);
} catch(err) {
  console.error(err);
}
```

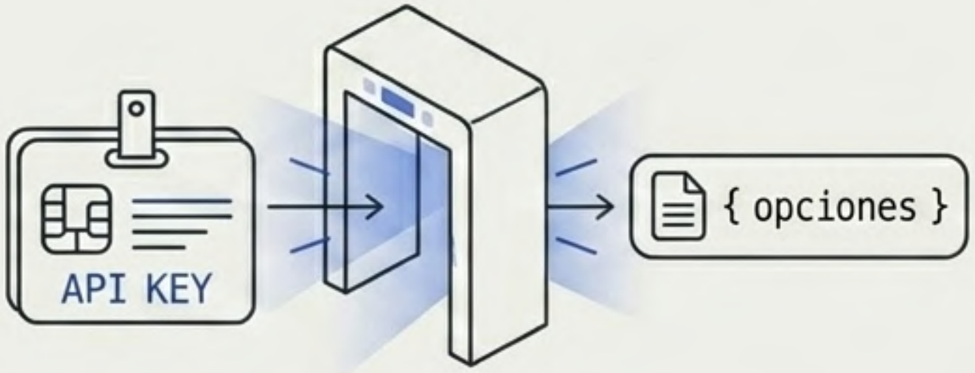
Request

JSON

Catch

NotebookLM

Identificándose: Headers y API Keys

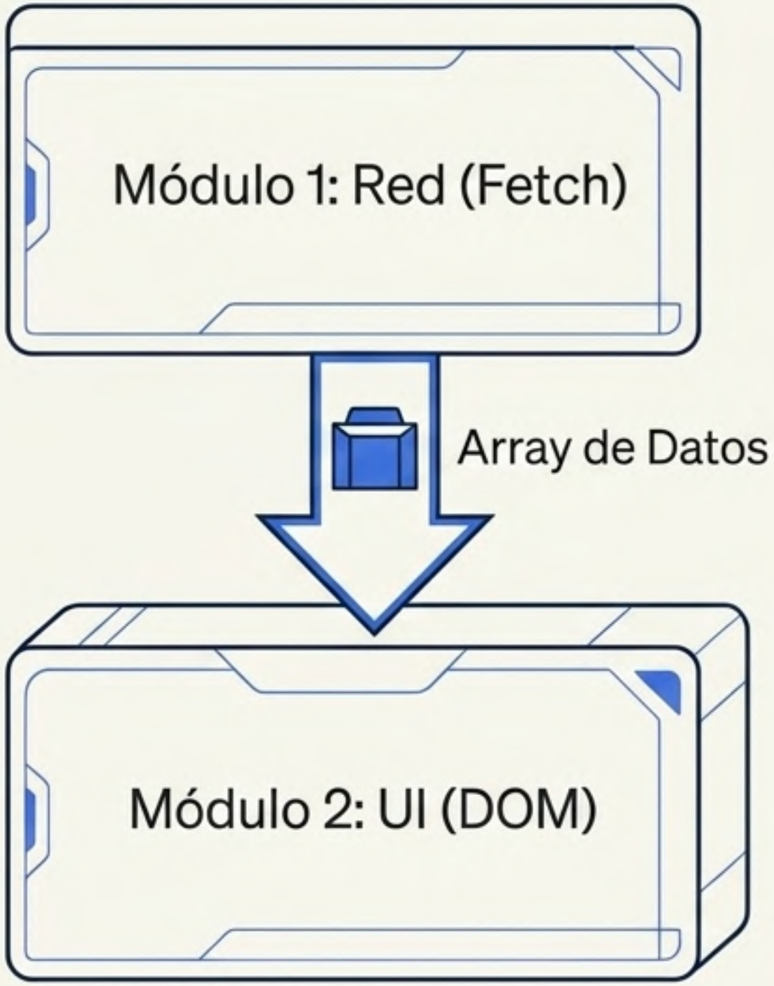



```
const opciones = {
  method: 'GET',
  headers: {
    'Authorization': 'Bearer MI_API_KEY_SECRETA',
    'Content-Type': 'application/json'
  }
};

const response = await fetch('https://api.privada.com/datos', opciones);
```

El header Authorization actúa como tu credencial de acceso. Nunca subas tu API Key a repositorios públicos.

NotebookLM

Buenas Prácticas: Separación de Responsabilidades




```
// 1. Solo obtiene datos
async function obtenerPeliculas() {
  const res = await fetch(url);
  return await res.json();
}
```

```
// 2. Solo dibuja en pantalla
function renderPeliculas(peliculas) {
  for (const peli of peliculas) {
    /* lógica de appendChild */
  }
}
```

NotebookLM

El Viaje **Completo**: Ejemplo Integrador

```
async function app() {
  const container = document.querySelector('#movies');
  try {
    const res = await fetch('https://api.movies.com/top');
    if (!res.ok) throw new Error(`HTTP Error: ${res.status}`);

    const movies = await res.json();

    for (const movie of movies) {
      const card = document.createElement('div');
      card.textContent = movie.title;
      container.appendChild(card);
    }
  } catch (error) {
    container.textContent = 'Error al cargar los datos.';
    console.error(error);
  }
}
app();
```

1. Petición
2. Validación Estricta
3. Parseo
4. Renderizado Seguro

NotebookLM

Lista de Verificación y Próximos Pasos

- ✓ Usa `fetch()` preferentemente con `async` / `await`.
- ✓ ¡SIEMPRE valida `response.ok` antes de continuar!
- ✓ Transforma la respuesta a formato utilizable (`await response.json()`).
- ✓ Usa `for...of` o `forEach` para iterar colecciones.
- ✓ Usa `createElement` y `textContent` (nunca `innerHTML` crudo).
- ✓ Separa tus funciones de red de tus funciones de UI.

Tu turno: Utiliza una API pública de usuarios, filtra la respuesta, y renderiza tarjetas en el DOM utilizando la arquitectura aprendida. ¡A codear!

NotebookLM